ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО
Научный руководитель,
доцент департамента
программной инженерии

_____ Р. А. Нестеров
«__» _____ 2026г.

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»
старший преподаватель департамента
программной инженерии

_____Н.А. Павлочев
«__» _____ 2026г.

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл	RU.17701729.05.15-01 51 01-1

Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.15-01 51 01-1-ЛУ

Исполнитель:
студент группы БПИ228
_____ / Д. Ю. Маслов/
«____»_____ 2026 г.

Москва 2026

УТВЕРЖДЕН
RU.17701729.10.01-01 51 01-1-ЛУ

Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть

Программа и методика испытаний

RU.17701729.10.01-01 51 01-1

Листов 44

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл	

ОГЛАВЛЕНИЕ

АННОТАЦИЯ *4*
1. ОБЪЕКТ ИСПЫТАНИЙ *6*
1.1. Наименование 6
1.2. Область применения 6
1.3. Обозначение испытуемой программы 6
2. ЦЕЛЬ ИСПЫТАНИЙ *7*
3. ТРЕБОВАНИЯ К ПРОГРАММЕ *8*
3.1. Требования к функциональным характеристикам 8
3.1.1. Требования к составу выполняемых функций 8
3.1.2. Требования к организации входных данных 15
3.1.3. Требования к организации выходных данных 22
3.2. Требования к интерфейсу 23
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ *24*
4.1. Состав программной документации 24
4.2. Специальные требования к программной документации 24
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ *26*
5.1. Технические средства, используемые во время испытаний 26
5.2. Программные средства, используемые во время испытаний 26
5.3. Порядок проведения испытаний 26
5.4. Условия проведения испытаний 26
5.4.1. Климатические условия 26
5.4.2. Требования к численности и квалификации персонала 27
6. МЕТОДЫ ИСПЫТАНИЙ *28*
6.1. Испытание выполнения требований к программной документации 29
6.2. Испытание функциональных требований 29
6.2.1. Модуль аутентификации (Auth Service) 29
6.2.1.1. Регистрация пользователя (POST /auth/register) 29
6.2.1.2. Аутентификация (POST /auth/login) 29
6.2.1.3. Обновление токенов (POST /auth/refresh) 30
6.2.1.4. Восстановление доступа (POST /auth/recover, POST /auth/reset-password) 30

6.2.2. Модуль аватаров (Avatar Service) 30
6.2.2.1. Генерация ссылки для загрузки (POST /avatars) 30
6.2.3. Модуль управления данными (CRUD Service) 31
6.2.3.1. Управление пользовательской информацией (User Info) 31
6.2.3.2. Управление параметрами пользователя (User Params) 31
6.2.3.3. Управление историей веса (User Weight) 32
6.2.3.4. Управление калориями (User Calories) 32
6.2.3.5. Управление пищевыми записями (Food Entries) 32
6.2.3.6. Управление упражнениями (Exercises) 33
6.2.3.7. Управление задачами (Tasks) 33
6.2.3.8. Управление связью тренировок и упражнений (Workout Exercises) 34
6.2.4. Модуль тренировок (Workout Service) 34
6.2.4.1. Генерация тренировки по запросу (POST /workouts) 34
6.2.4.2. Получение истории тренировок (GET /workouts/history) 34
6.2.4.3. Получение деталей тренировки (GET /workouts/{uuid}) 35
6.2.4.4. Обновление статуса тренировки (PATCH /workouts/{uuid}) 35
6.2.4.5. Удаление тренировки (DELETE /workouts/{uuid}) 35
6.2.5. Модуль питания (Nutrition Service) 35
6.2.5.1. Анализ фотографии еды (POST /nutrition/analyze) 35
6.2.6. Модуль рекомендаций (Recommendation Service) 35
6.2.6.1. Получение списка рекомендаций (GET /recommendations) 35
6.2.6.2. Отметка о прочтении (PATCH /recommendations/{uuid}/read) 36
6.2.6.3. Ручная генерация рекомендаций (POST /recommendations/refresh) 36
6.3. Испытание требований к надёжности 36
6.3.1. Отказоустойчивость при недоступности БД 36
6.3.2. Обработка повторных попыток в Executor Service 36
6.3.3. Защита от SQL-инъекций 36
6.3.4. JWT-безопасность 36
ПРИЛОЖЕНИЕ 1 37
ПРИЛОЖЕНИЕ 2 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 39

АННОТАЦИЯ

Программа и методика испытаний – это документ, в котором содержится информация о программном продукте, а также полное описание приемочных испытаний для данного программного продукта.

Настоящая Программа и методика испытаний для «Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть.» содержит следующие разделы:

1) «Объект испытаний»,
2) «Цель испытаний»,
3) «Требования к программе»,
4) «Требования к программным документам»,
5) «Средства и порядок испытаний»,
6) «Приложения».

В разделе «Объект испытаний» указано наименование и область применения разработанного продукта.

В разделе «Цель испытаний» указана цель проведения испытаний.

Раздел «Требования к программе» содержит основные требования к программе, которые подлежат проверке во время испытаний.

Раздел «Требования к программным документам» содержит состав программной документации, которая представляется на испытания, а также специальные требования к ней.

Раздел «Средства и порядок испытаний» содержит информацию о технических и программных средствах, которые следует использовать во время испытаний, а также порядок этих испытаний.

Раздел «Методы испытаний» содержит информацию об используемых методах испытаний.

Настоящий документ разработан в соответствии с требованиями:

1) ГОСТ 19.101-77 Виды программ и программных документов;
2) ГОСТ 19.102-77 Стадии разработки;
3) ГОСТ 19.103-77 Обозначения программ и программных документов;
4) ГОСТ 19.104-78 Основные надписи;
5) ГОСТ 19.105-78 Общие требования к программным документам;
6) ГОСТ 19.106-78 Требования к программным документам, выполненным печатным способом;

7) ГОСТ 19.301-79 Программа и методика испытаний. Требования к содержанию и оформлению.

Изменения к данному документу оформляются согласно ГОСТ 19.603-7, ГОСТ 19.604- 78.

Перед прочтением данного документа рекомендуется ознакомиться с терминологией, приведенной в Приложении 1 настоящей программы и методики испытаний.

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование

Наименование программы – Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть.

Наименование программы на английском языке – «AI-Powered Health and Fitness App with Personalized Nutrition and Training Plans. Backend».

Краткое наименование программы – «BodyFuel».

1.2. Область применения

Серверная часть платформы BodyFuel представляет собой централизованное ядро интеллектуальной фитнес-экосистемы, предназначенной для автоматизации процессов планирования физической активности и контроля питания пользователей. Система ориентирована на широкий круг лиц, стремящихся к здоровому образу жизни, но не имеющих достаточных знаний или времени для самостоятельного составления тренировочных программ и ведения дневника питания.

1.3. Обозначение испытуемой программы

Наименование темы разработки – Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть.

2. ЦЕЛЬ ИСПЫТАНИЙ

Цель проведения испытаний заключается в проверке соответствия характеристик разработанной программы функциональным требованиям и отдельным требованиям к надежности, которые были описаны в Техническом задании для данной программы.

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Требования к составу выполняемых функций

Сервис аутентификации (Auth Service) должен реализовывать следующие функции:

1. Регистрация нового пользователя с указанием корректных персональных данных, включающих в себя: логин (username), пароль (password), адрес электронной почты (email), имя (name), фамилию (surname), номер телефона (phone).
2. Аутентификация пользователя по логину и паролю с выдачей пары JWT-токенов: access-токена (короткоживущий) и refresh-токена (долгоживущий).
3. Обновление пары access/refresh токенов через отдельный эндпоинт с использованием действующего refresh-токена.
4. Верификация email и телефона пользователя путём отправки кода подтверждения и его проверки.
5. Восстановление доступа к аккаунту через привязанный email или телефон:
   5.1. инициирование восстановления (отправка кода или ссылки);
   5.2. сброс пароля с использованием полученного кода/токена
6. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   6.1. POST /auth/register – регистрация нового пользователя.
   6.2. POST /auth/login – аутентификация пользователя.
   6.3. POST /auth/refresh – обновление пары токенов.
   6.4. POST /auth/verify-email – подтверждение email.
   6.5. POST /auth/verify-phone – подтверждение телефона.
   6.6. POST /auth/recover – инициирование восстановления доступа.
   6.7. POST /auth/reset-password – сброс пароля с использованием токена.

Сервис управления данными (CRUD Service) должен реализовывать следующие функции:

Управление справочником упражнений (Exercises):

1. Хранение каталога упражнений с атрибутами: название, описание, уровень подготовки (beginner/medium/sportsman), тип упражнения (cardio/upper_body/lower_body/full_body/flexibility), место выполнения (home/gym/street),

базовое количество повторений, количество подходов, средние затраты калорий, базовое время отдыха, ссылка на GIF-демонстрацию.

2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /exercises – получение списка упражнений с возможностью фильтрации по параметрам: level_preparation, type_exercise, place_exercise.
   2.2. POST /exercises – создание нового упражнения.
   2.3. GET /exercises/{uuid} – получение упражнения по идентификатору.
   2.4. PATCH /exercises/{uuid} – обновление существующего упражнения (частичное обновление).
   2.5. DELETE /exercises/{uuid} – удаление упражнения.

Управление пользовательской информацией (User Info):

1. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   1.1. GET /user/info – получение информации о текущем пользователе (username, email, name, surname, phone, created_at).
   1.2. PATCH /user/info – обновление пользовательских данных (имя, фамилия, email, телефон).
   1.3. DELETE /user/info – удаление учётной записи пользователя (деактивация).

Управление параметрами пользователя (User Params):

1. Хранение параметров пользователя: рост, уровень повседневной активности (lifestyle), цель (wants), целевой вес, целевая суточная калорийность, целевое количество тренировок в неделю, ссылка на фото профиля
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /user/params – получение параметров пользователя (включая текущий вес, вычисляемый из истории).
   2.2. POST /user/params – создание параметров пользователя (все обязательные поля).
   2.3. PATCH /user/params – обновление параметров пользователя (частичное обновление).
   2.4. DELETE /user/params – удаление параметров пользователя.

Управление историей веса (User Weight):

1. Ведение временного ряда записей веса пользователя.
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /user/weight – получение последней записи веса пользователя.
   2.2. POST /user/weight – создание новой записи веса (значение веса, дата фиксируется автоматически).
   2.3. GET /user/weight/history – получение полной истории изменений веса пользователя (с пагинацией).
   2.4. DELETE /user/weight/{uuid} – удаление конкретной записи веса по идентификатору.

Управление калориями (User Calories):

1. Учёт дополнительных калорий (потребление/затраты) вне детального дневника питания.
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /user/calories/history – получение истории записей о калориях за период (с пагинацией)
   2.2. POST /user/calories – создание записи о потреблённых или затраченных калориях (calories, date, description опционально).
   2.3. PATCH /user/calories/{uuid} – обновление записи о калориях.
   2.4. DELETE /user/calories/{uuid} – удаление записи о калориях.

Управление пищевыми записями (Food Entries):

1. Ведение дневника питания с детализацией по приёмам пищи.
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /nutrition/diary – получение дневника питания за указанную дату (суммарные показатели и список приёмов пищи).
   2.2. POST /nutrition/entries – создание записи о приёме пищи (description, calories, protein, carbs, fat, meal_type, photo_url опционально).
   2.3. PATCH /nutrition/entries/{uuid} – обновление записи о приёме пищи.
   2.4. DELETE /nutrition/entries/{uuid} – удаление записи о приёме пищи.

2.5. GET /nutrition/report – получение отчёта по питанию за выбранный период (агрегированные данные).

Управление задачами (Tasks):

1. Хранение и управление отложенными задачами (тип, статус, попытки, атрибуты).
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /tasks – получение списка задач с фильтрацией по типу, статусу, пользователю (с пагинацией).
   2.2. GET /tasks/{uuid} – получение детальной информации о задаче по идентификатору.
   2.3. PATCH /tasks/{uuid} – обновление задачи (например, изменение статуса, сброс попыток).
   2.4. DELETE /tasks/{uuid} – удаление задачи.
   2.5. POST /tasks/{uuid}/retry – принудительный перезапуск задачи.

Управление связью тренировок и упражнений (Workout Exercises):

1. Отслеживание выполнения каждого упражнения в рамках конкретной тренировки.
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. GET /workouts/{workoutId}/exercises – получение списка упражнений для конкретной тренировки с их статусами
   2.2. POST /workouts/{workoutId}/exercises – добавление упражнения в тренировку (с параметрами выполнения).
   2.3. PATCH /workouts/exercises/{uuid} – обновление информации о выполнении упражнения (подходы, повторения, вес, статус).
   2.4. DELETE /workouts/exercises/{uuid} – удаление упражнения из тренировки.

Сервис тренировок (Workout Service) должен реализовывать следующие функции:

1. Автоматическая генерация персонализированных тренировок на основе цели пользователя, уровня подготовки, истории тренировок и доступного оборудования.

2. Периодическая фоновая генерация тренировок с проверкой наличия активных незавершённых тренировок и соблюдения времени отдыха.
3. Учёт предпочтений пользователя (популярные типы упражнений и места тренировок) на основе анализа истории.
4. Применение принципа прогрессивной нагрузки с увеличением сложности при успешном выполнении серии тренировок.
5. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   5.1. POST /workouts – генерация тренировки по запросу пользователя с возможностью указания параметров (exercises_count, level, place_exercise, type_exercise). Если параметры не указаны, генерация происходит на основе профиля.
   5.2. GET /workouts/history – получение истории тренировок пользователя (с фильтрацией по статусу, дате, уровню сложности и пагинацией).
   5.3. GET /workouts/{uuid} – получение детальной информации о тренировке по идентификатору, включая список упражнений с их статусами.
   5.4. PATCH /workouts/{uuid} – отметка о выполнении тренировки (указание статуса done/failed, фактических калорий и времени).
   5.5. DELETE /workouts/{uuid} – удаление тренировки.
6. Создание отложенной задачи на уведомление о готовности новой тренировки.
7. Расчёт прогнозируемых энергозатрат и продолжительности для каждой сгенерированной тренировки.

Сервис питания (Nutrition Service) должен реализовывать следующие функции:

1. Загрузка фотографий еды в S3-бакет и их анализ через OpenAI API для определения пищевой ценности (КБЖУ).
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. POST /nutrition/analyze – загрузка фотографии еды (multipart/form-data) и получение результата анализа (синхронно или асинхронно с возвратом task_id).
   2.2. GET /nutrition/analysis/{taskId} – получение результата асинхронного анализа по идентификатору задачи.
   2.3. POST /nutrition/entries – ручное создание записи о приёме пищи.
   2.4. GET /nutrition/diary – получение дневника питания за день.

2.5. GET /nutrition/report – получение отчёта за период.

3. Периодическая фоновая проверка баланса питания для пользователей и создание отложенных задач на уведомление при значительных отклонениях от нормы.

Сервис рекомендаций (Recommendation Service) должен реализовывать следующие функции:

1. Фоновая генерация персонализированных ежедневных рекомендаций по питанию, тренировкам и восстановлению на основе цели пользователя и его текущего прогресса.
2. Контекстно-зависимые советы.
3. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   3.1. GET /recommendations – получение списка активных рекомендаций для текущего пользователя (с пагинацией и фильтрацией по статусу прочтения).
   3.2. POST /recommendations/refresh – ручная генерация рекомендаций по запросу пользователя.
   3.3. PATCH /recommendations/{uuid}/read – отметка рекомендации как прочитанной
4. Интеграция с OpenAI API для расширенной генерации текстовых советов.
5. Управление историей выданных рекомендаций.

Сервис аватаров (Avatar Service) должен реализовывать следующие функции:

1. Обеспечение возможности обновления или удаления ссылки на аватар в профиле пользователя (через эндпоинты управления параметрами пользователя).
2. Имплементация поведения и корректные ответы на HTTP-запросы по путям:
   2.1. POST /avatars – генерация подписанного URL для прямой загрузки фотографии в S3-хранилище на основе MIME-типа файла. В ответ возвращаются: upload_url (URL для PUT-запроса), object_key (ключ объекта), avatar_url.

Сервис отложенных задач (Executor Service) должен реализовывать следующие функции:

1. Получение и обработка отложенных задач на отправку уведомлений (push, SMS, email) и выполнение других фоновых операций (генерация тренировок, анализ фотографий, проверка баланса питания, генерация рекомендаций).
2. Управление очередью задач с поддержкой повторных попыток отправки при неудаче (с настраиваемым лимитом попыток и экспоненциальной задержкой)

3. Интеграция с внешними провайдерами (для отправки SMS – Twilio, для отправки push-уведомлений Apple Push Notification Service (APNS)).
4. Планирование периодических задач (ежедневная генерация рекомендаций, напоминания о тренировках, очистка старых задач).

3.1.2. Требования к организации входных данных

Все входные данные передаются в формате JSON, если не указано иное. Файлы (изображения) передаются как multipart/form-data. Даты и время передаются в формате ISO 8601 (например, «2026-03-20T10:30:00Z»). Идентификаторы ресурсов имеют формат UUID (v4).

Ниже приведены требования к входным данным для каждого эндпоинта.

1. Сервис аутентификации (Auth Service):
   - 1.1. POST /auth/register:
     - 1.1.1. Тело запроса (JSON):
       - 1.1.1.1. username (string, обязательный, 3-32 символа) – уникальное имя пользователя
       - 1.1.1.2. password (string, обязательный, минимум 6 символов) – пароль.
       - 1.1.1.3. email (string, обязательный) – адрес электронной почты (проверка формата).
       - 1.1.1.4. name (string, обязательный, 2-50 символов) – имя.
       - 1.1.1.5. surname (string, обязательный, 2-50 символов) – фамилия.
       - 1.1.1.6. phone (string, обязательный) – номер телефона в международном формате
   - 1.2. POST /auth/login
     - 1.2.1. Тело запроса (JSON):
       - 1.2.1.1. username (string, обязательный) – логин
       - 1.2.1.2. password (string, обязательный) – пароль
   - 1.3. POST /auth/refresh
     - 1.3.1. Заголовок Authorization: Bearer <refresh_token> (refresh-токен, выданный ранее)
     - 1.3.2. Тело запроса отсутствует
   - 1.4. POST /auth/verify-email
     - 1.4.1. Тело запроса (JSON)
       - 1.4.1.1. code (string, обязательный) – код подтверждения, отправленный на email
   - 1.5. POST /auth/verify-phone
     - 1.5.1. Тело запроса (JSON)
       - 1.5.1.1. code (string, обязательный) – код подтверждения, отправленный по SMS
   - 1.6. POST /auth/recover

1.6.1. Тело запроса (JSON) – одно из полей

1.6.1.1. email (string, опционально) – email для восстановления

1.6.1.2. phone (string, опционально) – телефон для восстановления

1.7. POST /auth/reset-password

1.7.1. Тело запроса (JSON)

1.7.1.1. token (string, обязательный) – токен сброса пароля (полученный по email/SMS)

1.7.1.2. new_password (string, обязательный, минимум 6 символов) – новый пароль

2. Сервис аватаров (Avatar Service):

2.1. POST /avatars:

2.1.1. Тело запроса (JSON)

2.1.1.1. content_type (string, обязательный) – MIME-тип загружаемого файла (например, «image/jpeg», «image/png»)

3. Управление пользовательской информацией (User Info)

3.1. GET /user/info

3.1.1. Параметры: отсутствуют (идентификатор пользователя извлекается из JWT)

3.2. PATCH /user/info

3.2.1. Тело запроса (JSON, частичное обновление)

3.2.1.1. name (string, опционально, 2-50 символов)

3.2.1.2. surname (string, опционально, 2-50 символов)

3.2.1.3. email (string, опционально)

3.2.1.4. phone (string, опционально)

3.3. DELETE /user/info

3.3.1. Параметры: отсутствуют

4. Управление параметрами пользователя (User Params)

4.1. GET /user/params

4.1.1. Параметры: отсутствуют

4.2. POST /user/params

4.2.1. Тело запроса (JSON)

4.2.1.1. height (integer, обязательный, 100-250) – рост в см

4.2.1.2. lifestyle (string, обязательный, enum: «not_active», «active», «sportive») – образ жизни

4.2.1.3. wants (string, обязательный, enum: «lose_weight», «build_muscle», «stay_fit») – цель

4.2.1.4. targetWeight (number, обязательный, 40-300) – целевой вес (кг)

4.2.1.5. targetCaloriesDaily (integer, обязательный, 0-10000) – целевая суточная калорийность

4.2.1.6. targetWorkoutsWeeks (integer, обязательный, 0-7) – целевое количество тренировок в неделю

4.2.1.7. photo (string, опционально) – URL аватара (после загрузки)

4.3. PATCH /user/params

4.3.1. Тело запроса (JSON)

4.3.1.1. любые поля из POST, все опциональны, с теми же ограничениями

4.4. DELETE /user/params

4.4.1. Параметры: отсутствуют

5. Управление историей веса (User Weight)

5.1. GET /user/weight

5.1.1. Параметры: отсутствуют

5.2. POST /user/weight

5.2.1. Тело запроса (JSON)

5.2.1.1. weight (number, обязательный, 10-300) – вес в кг

5.3. GET /user/weight/history

5.3.1. Параметры запроса (опционально)

5.3.1.1. limit (integer, по умолчанию 20, максимум 100) – количество записей на странице

5.4. DELETE /user/weight/{uuid}

5.4.1. Параметр пути

5.4.1.1. uuid (string, обязательный) – идентификатор записи веса

6. Управление калориями (User Calories)

6.1. GET /user/calories/history

6.1.1. Параметры запроса (опционально):

6.1.1.1. start_date (string, ISO 8601) – начальная дата

6.1.1.2. end_date (string, ISO 8601) – конечная дата

6.1.1.3. page, limit (аналогично весу)

6.2. POST /user/calories

6.2.1. Тело запроса (JSON)

6.2.1.1. calories (integer, обязательный) – количество калорий

6.2.1.2. date (string, обязательный, ISO 8601) – дата и время

6.2.1.3. description (string, опционально, макс. 255 символов) – комментарий

6.3. PATCH /user/calories/{uuid}

6.3.1. Параметр пути: uuid

6.3.2. Тело запроса (JSON)

6.3.2.1. любые поля из POST, все опциональны

6.4. DELETE /user/calories/{uuid}

6.4.1. Параметр пути: uuid

7. Управление пищевыми записями (Food Entries)

7.1. GET /nutrition/diary

7.1.1. Параметры запроса

7.1.1.1. date (string, обязательный, YYYY-MM-DD) – дата, за которую запрашивается дневник

7.2. POST /nutrition/entries

7.2.1. Тело запроса (JSON)

7.2.1.1. description (string, обязательный, макс. 255 символов) – описание блюда

7.2.1.2. calories (integer, обязательный, 0-10000) – калорийность

7.2.1.3. protein (number, обязательный, 0-1000) – белки (г)

7.2.1.4. carbs (number, обязательный, 0-1000) – углеводы (г)

7.2.1.5. fat (number, обязательный, 0-1000) – жиры (г)

7.2.1.6. meal_type (string, опционально, enum: «breakfast», «lunch», «dinner», «snack») – тип приёма пищи

7.2.1.7. photo_url (string, опционально) – ссылка на фотографию (если загружалась)

7.3. PATCH /nutrition/entries/{uuid}

7.3.1. Параметр пути: uuid

7.3.2. Тело запроса (JSON)

7.3.2.1. любые поля из POST, все опциональны

7.4. DELETE /nutrition/entries/{uuid}

7.4.1. Параметр пути: uuid

7.5. GET /nutrition/report

7.5.1. Параметры запроса

7.5.1.1. start_date (string, обязательный, YYYY-MM-DD) – начало периода

7.5.1.2. end_date (string, обязательный, YYYY-MM-DD) – конец периода

8. Управление задачами (Tasks)

8.1. GET /tasks

8.1.1. Параметры запроса (опционально)

8.1.1.1. type (string, enum: «email», «sms», «push», «workout_generation», «nutrition_analysis») – тип задачи

8.1.1.2. state (string, enum: «pending», «processing», «completed», «failed») – статус

8.1.1.3. user_id (uuid) – идентификатор пользователя

8.2. GET /tasks/{uuid}

8.2.1. Параметр пути: uuid

8.3. PATCH /tasks/{uuid}

8.3.1. Параметр пути: uuid

8.3.2. Тело запроса (JSON)

8.3.2.1. state (string, опционально) – новый статус

8.3.2.2. attempts (integer, опционально) – сброс/изменение счётчика попыток

8.4. DELETE /tasks/{uuid}

8.4.1. Параметр пути: uuid

8.5. POST /tasks/{uuid}/retry

8.5.1. Параметр пути: uuid

8.5.2. Тело запроса отсутствует

9. Управление упражнениями (Exercises)

9.1. GET /exercises

9.1.1. Параметры запроса (все опциональны)

9.1.1.1. level_preparation (string, enum: «beginner», «medium», «sportsman»)

9.1.1.2. type_exercise (string, enum: «cardio», «upper_body», «lower_body», «full_body», «flexibility»)

9.1.1.3. place_exercise (string, enum: «home», «gym», «street»)

9.2. POST /exercises

9.2.1. Тело запроса (JSON)

9.2.1.1. name (string, обязательный, 1-100 символов)

9.2.1.2. description (string, опционально, макс. 1000 символов)

9.2.1.3. level_preparation (string, обязательный, enum)

9.2.1.4. type_exercise (string, обязательный, enum)

9.2.1.5. place_exercise (string, обязательный, enum)

9.2.1.6. base_count_reps (integer, обязательный, 1-1000)

9.2.1.7. steps (integer, обязательный, 1-100)

9.2.1.8. avg_calories_per (number, обязательный, 0-1000)

9.2.1.9. base_relax_time (integer, обязательный, 0-3600)

9.2.1.10. link_gif (string, опционально) – URL GIF-демонстрации

9.3. GET /exercises/{uuid}

9.3.1. Параметр пути: uuid

9.4. PATCH /exercises/{uuid}

9.4.1. Параметр пути: uuid

9.4.2. Тело запроса (JSON)

9.4.2.1. любые поля из POST, все опциональны.

9.5. DELETE /exercises/{uuid}

9.5.1. Параметр пути: uuid

10. Управление связью тренировок и упражнений (Workout Exercises)

10.1. GET /workouts/{workoutId}/exercises

10.1.1. Параметр пути

10.1.1.1. workoutId (uuid, обязательный)

10.2. POST /workouts/{workoutId}/exercises

10.2.1. Параметр пути: workoutId

10.2.2. Тело запроса (JSON)

10.2.2.1. exercise_id (uuid, обязательный)

10.2.2.2. modify_reps (integer, опционально) – изменённое количество повторений

10.2.2.3. modify_relax_time (integer, опционально) – изменённое время отдыха

10.3. PATCH /workouts/exercises/{uuid}

10.3.1. Параметр пути: uuid

10.3.2. Тело запроса (JSON)

10.3.2.1. status (string, опционально, enum: «pending», «in_progress», «completed», «skipped»)

10.3.2.2. actual_reps (integer, опционально)

10.3.2.3. actual_weight (number, опционально)

10.3.2.4. completed_at (string, опционально, ISO 8601)

10.4. DELETE /workouts/exercises/{uuid}

10.4.1. Параметр пути: uuid

11. Сервис тренировок (Workout Service)

11.1. POST /workouts

11.1.1. Тело запроса (JSON) – все поля опциональны

11.1.1.1. exercises_count (integer, 4-20) – желаемое количество упражнений

11.1.1.2. level (string, enum: «workout_light», «workout_middle», «workout_hard») – желаемый уровень сложности

11.1.1.3. place_exercise (string, enum: «home», «gym», «street») – предпочтительное место

11.1.1.4. type_exercise (string, enum) – предпочтительный тип упражнений. Если параметры не указаны, генерация происходит на основе профиля пользователя

11.2. GET /workouts/history

11.2.1. Параметры запроса (опционально)

11.2.1.1. status (string, enum: «workout_created», «workout_in_active», «workout_done», «workout_failed»)

11.2.1.2. start_date, end_date (ISO 8601)

11.3. GET /workouts/{uuid}

11.3.1. Параметр пути: uuid

11.4. PATCH /workouts/{uuid}

11.4.1. Параметр пути: uuid

11.4.2. Тело запроса (JSON)

11.4.2.1. status (string, опционально, enum: «workout_done», «workout_failed»)

11.4.2.2. total_calories (integer, опционально) – фактические затраты калорий

11.4.2.3. duration (integer, опционально) – фактическая длительность в секундах

11.5. DELETE /workouts/{uuid}

11.5.1. Параметр пути: uuid

12. Сервис питания (Nutrition Service)

12.1. POST /nutrition/analyze:

12.1.1. Формат: multipart/form-data

12.2. GET /nutrition/analysis/{taskId}

12.2.1. Параметр пути

12.2.1.1. taskId (uuid, обязательный) – идентификатор задачи асинхронного анализа

13. Сервис рекомендаций (Recommendation Service)

13.1. GET /recommendations

13.1.1. Параметры запроса (опционально)

13.1.1.1. is_read (boolean) – фильтр по статусу прочтения

13.2. POST /recommendations/refresh

13.2.1. Тело запроса отсутствует

13.3. PATCH /recommendations/{uuid}/read

13.3.1. Параметр пути: uuid

13.3.2. Тело запроса отсутствует

3.1.3. Требования к организации выходных данных

Система обеспечивает унифицированный и структурированный обмен данными между клиентом и сервером. Ответы аутентификации и управления пользователями содержат JWT-токены доступа и обновления с указанием времени экспирации. Вместе с токенами возвращаются профильные данные пользователя в нормализованном формате, включая вычисленные производные показатели, а также статусы успешного выполнения операций изменения данных.

Сгенерированные планы тренировок передаются в виде иерархических структур. Они включают метаданные (тип плана, общая продолжительность, уровень сложности) и детализированные списки упражнений. Для каждой позиции указываются название, целевые группы мышц, параметры подходов (количество, повторения, время отдыха), а также визуальные ссылки на технику выполнения с возможностью пропуска упражнения.

Результаты анализа питания формируются как комплексные объекты. Они содержат распознанные продукты (наименование, уверенность распознавания, координаты на изображении), расчетную пищевую ценность (калории, белки, жиры, углеводы на 100 грамм и для указанной порции), а также варианты замены и рекомендации по балансировке приема пищи.

Данные мониторинга и прогресса организуются в агрегированные временные ряды. Система отслеживает показатели активности (количество тренировок за неделю, калорийность, вес), статистику выполнения планов (количество завершенных тренировок,

соблюдение целей по корректировке веса) и предоставляет сравнительную аналитику динамики изменений с формированием трендов и персонализированных рекомендаций.

Рекомендательные выводы AI структурируются как тематические блоки с четкой классификацией по типам (тренировочные, нутрициональные, мотивационные). Каждый блок содержит приоритет актуальности, обоснование выдачи и конкретные советы с параметризацией по времени выполнения и ожидаемому эффекту.

Все ответы системы, включая вышеперечисленные, следуют единому формату с обязательными полями: статус, временная метка, код результата и детализированное сообщение. Для успешных операций предоставляется подтверждение выполнения, а для ошибочных сценариев возвращается иерархия ошибок с классификацией по типам (клиентские, серверные, внешние сервисы), рекомендациями по устранению и техническими деталями для отладки.

Остальные запросы не имеют дополнительных требований к выходным данным и обрабатываются в стандартном режиме.

3.2. Требования к интерфейсу

Интерфейсы сервисов должны соответствовать нижеперечисленным требованиям:

1. воспроизводить функционал из пункта 3.1.1
2. показывать минимально необходимую информацию касательно статуса выполнения операции
3. подсказывать оператору, в каком месте допущена ошибка

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

4.1. Состав программной документации

На испытания должна быть представлена документация к программе в следующем составе:

1) «Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть». Техническое задание (ГОСТ 19.201-78)
2) «Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть». Программа и методика испытаний (ГОСТ 19.301-78)
3) «Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть». Текст программы (ГОСТ 19.401-78)
4) «Приложение для персонализированного контроля питания и тренировок с использованием AI и интеграцией с данными о здоровье. Серверная часть». Руководство оператора (ГОСТ 19.505-79)

4.2. Специальные требования к программной документации

1) Все документы к программе должны быть выполнены в соответствии с ГОСТ 19.106-78 и ГОСТ к этому виду документа (см. п. 5.1.).
2) Пояснительная записка должна быть загружена в систему Антиплагиат через ЛМС НИУ ВШЭ. Лист, подтверждающий загрузку пояснительной записки, сдается в учебный офис вместе со всеми материалами не позже, чем за день до защиты курсовой работы.
3) Вся документация сдается в печатном виде, при этом она должна быть обязательно подписана академическим руководителем образовательной программы 09.03.04 «Программная инженерия», руководителем разработки и исполнителем перед сдачей курсовой работы в учебный офис не позже одного дня до защиты.
4) Вся документация и программа также сдается в электронном виде в формате .pdf или .docx. в архиве формата .rar или .zip.

5) Все документы перед защитой ВКР должны быть загружены в информационно-образовательную среду НИУ ВШЭ LMS (Learning management system) в личном кабинете, дисциплина - «Дипломное проектирование», одним архивом (см. п.3).

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства, используемые во время испытаний

Для надёжной и бесперебойной работы программы требуется следующий состав технических средств:

1. процессор с тактовой частотой не менее 1 Ггц
2. оперативная память не менее 1 ГБ
3. жесткий диск со свободным местом не менее 3 ГБ
4. монитор и HDMI кабель
5. компьютерная мышь
6. операционная система Windows 8 и выше, macOS или linux

5.2. Программные средства, используемые во время испытаний

Для работы программы необходим следующий состав программных средств:

1. Операционная система Windows 8 и выше, macOS или linux
2. Браузер с поддержкой языка разметки HTML5
3. Интернет-соединение со скоростью не менее 60 Кб\сек

5.3. Порядок проведения испытаний

1) проверка требований к программной документации;

2) проверка требований к интерфейсу;

3) проверка требований к надежности;

4) проверка требований к функциональным характеристикам.

5.4. Условия проведения испытаний

5.4.1. Климатические условия

Климатические условия эксплуатации, при которых должны обеспечиваться заданные характеристики, должны удовлетворять требованиям, предъявляемым к персональным компьютерам в части условий их эксплуатации.

Персональный компьютер предназначен для работы в закрытом отапливаемом помещении со стабильными климатическими условиями категории 4.1 согласно ГОСТ 15150-69.

5.4.2. Требования к численности и квалификации персонала

Минимальное количество персонала, требуемого для работы программы, должно составлять не менее 1 штатной единицы:

1) конечный пользователь – оператор ЭВМ.

Оператор ЭВМ должен:

1) обладать практическими навыками работы с пользовательским интерфейсом операционной системы

2) обладать установленным Docker на ЭВМ

6. МЕТОДЫ ИСПЫТАНИЙ

Испытания проводятся методом функционального тестирования «чёрного ящика» – отправка HTTP-запросов к API с контролем кодов ответа и структуры возвращаемых данных. В ходе таких проверок контролируются не только HTTP-статусы ответов (например, 200 OK, 201 Created, 400 Bad Request), но и структура возвращаемых данных, их типы и соответствие ожидаемой схеме JSON. Каждый тестовый сценарий при этом оформляется в виде документа, где заранее фиксируется описание входных данных и предполагаемый результат, а в процессе выполнения тестов вносятся фактические результаты, что позволяет наглядно сравнивать ожидание и реальность.

Особое внимание уделяется проверке асинхронных операций, которые не дают мгновенного ответа с конечным результатом. В таких случаях применяются два основных подхода: либо организован опрос статуса выполнения задачи через специальные эндпоинты, пока статус не сменится на «успешно» или «ошибка», либо выполняется прямая проверка наличия созданных сущностей в базе данных, что служит косвенным подтверждением завершения фонового процесса. Такой подход позволяет гарантировать корректность работы функционала, который обрабатывается очередями или длительными вычислениями.

Прежде чем приступить к непосредственному выполнению тестов, необходимо провести комплекс подготовительных работ по развёртыванию инфраструктуры. Весь стек сервисов поднимается с помощью Docker Compose, что обеспечивает воспроизводимость окружения. В состав инфраструктуры входят: база данных PostgreSQL для хранения основного состояния приложения, Redis для кэширования и управления очередями, MinIO в качестве S3-совместимого хранилища для файлов и статики, а также само тестируемое приложение, сконфигурированное для работы с этими зависимостями.

После запуска контейнеров следующим обязательным шагом является применение миграций базы данных — они создают необходимые таблицы, индексы и связи. Затем, если того требует функциональность, производится загрузка начальных данных: например, наполнение справочника упражнений, предустановленного списка флагов или иных статических сущностей, без которых приложение не сможет функционировать в полном объёме либо тесты будут нерепрезентативны. Финальным этапом подготовки становится проверка доступности всех сервисов и эндпоинтов: обычно для этого используются встроенные механизмы health check (проверки состояния), которые позволяют убедиться, что

PostgreSQL принимает соединения, Redis отвечает на ping, MinIO готов к операциям, а само приложение возвращает корректный статус на служебных маршрутах. Только после прохождения всех этих шагов среда считается готовой к проведению тестирования.

6.1. Испытание выполнения требований к программной документации

Состав программной документации проверяется визуально, проверяется наличие всех подписей и наличие программной документации в системе LMS. Также визуально проверяется соответствие документации требованиям ГОСТ. Все документы удовлетворяют представленным требованиям.

6.2. Испытание функциональных требований

6.2.1. Модуль аутентификации (Auth Service)

6.2.1.1. Регистрация пользователя (POST /auth/register)

1. Успешная регистрация.
    i. Входные данные: корректные поля username, password, email, name, surname, phone
    ii. Ожидаемый результат: код 201, сообщение об успехе. В базе данных появляется запись в таблице user_info с хешированным паролем.
2. Попытка регистрации с уже существующим username
    i. Входные данные: username уже зарегистрирован
    ii. Ожидаемый результат: код 409 Conflict, тело с описанием ошибки
3. Попытка регистрации с невалидным email
    i. Входные данные: email без символа '@'
    ii. Ожидаемый результат: код 400 Bad Request
4. Попытка регистрации с коротким паролем (менее 6 символов)
    i. Ожидаемый результат: код 400
5. Отсутствие обязательного поля (например, username)
    i. Ожидаемый результат: код 400

6.2.1.2. Аутентификация (POST /auth/login)

1. Успешный вход
    i. Входные данные: верные username и password

ii. Ожидаемый результат: код 200, тело содержит access_token, refresh_token, expires_in

2. Неверный пароль
   i. Ожидаемый результат: код 401 Unauthorized
3. Несуществующий пользователь
   i. Ожидаемый результат: код 401 (для безопасности сообщение одинаковое)

6.2.1.3. Обновление токенов (POST /auth/refresh)

1. Успешное обновление
   i. Входные данные: валидный refresh-токен в заголовке Authorization
   ii. Ожидаемый результат: код 200, новая пара токенов. Старый refresh-токен становится недействительным
2. Попытка обновления с недействительным refresh-токеном
   i. Ожидаемый результат: код 401
3. Попытка обновления без токена
   i. Ожидаемый результат: код 401

6.2.1.4. Восстановление доступа (POST /auth/recover, POST /auth/reset-password)

1. Успешное инициирование восстановления по email
   i. Входные данные: { "email": "user@example.com" }
   ii. Ожидаемый результат: код 200 (всегда одинаковый, даже если email не найден)
2. Успешный сброс пароля
   i. Входные данные: { "token": "valid_token", "new_password": "newPass123" }
   ii. Ожидаемый результат: код 200, пароль изменён
3. Попытка сброса с недействительным токеном
   i. Ожидаемый результат: код 400

6.2.2. Модуль аватаров (Avatar Service)

6.2.2.1.Генерация ссылки для загрузки (POST /avatars)

1. Успешная генерация
   i. Входные данные: { "content_type": "image/jpeg" }

ii. Ожидаемый результат: код 200, тело содержит upload_url, object_key, avatar_url. URL подписан и позволяет загрузить файл методом PUT

2. Неподдерживаемый content_type
   i. Входные данные: { "content_type": "image/gif" } (если GIF не поддерживаются)
   ii. Ожидаемый результат: код 400
3. Без авторизации
   i. Ожидаемый результат: код 401

6.2.3. Модуль управления данными (CRUD Service)

6.2.3.1. Управление пользовательской информацией (User Info)

1. Получение информации о текущем пользователе (GET /user/info)
   i. Входные данные: валидный токен
   ii. Ожидаемый результат: код 200, JSON с данными пользователя (username, email, name, surname, phone, created_at)
2. Попытка получения без токена
   i. Ожидаемый результат: код 401
3. Обновление имени (PATCH /user/info)
   i. Входные данные: { "name": "НовоеИмя" }
   ii. Ожидаемый результат: код 204. При повторном GET имя изменилось
4. Обновление email на уже существующий
   i. Входные данные: email другого пользователя
   ii. Ожидаемый результат: код 409 Conflict

6.2.3.2. Управление параметрами пользователя (User Params)

1. Создание параметров (POST /user/params)
   i. Входные данные: полный JSON со всеми обязательными полями
   ii. Ожидаемый результат: код 201
2. Создание с пропуском обязательного поля
   i. Ожидаемый результат: код 400
3. Получение параметров (GET /user/params)
   i. Ожидаемый результат: код 200, все поля, включая currentWeight
4. Обновление параметров (PATCH /user/params)

i. Входные данные: { "height": 185 }
ii. Ожидаемый результат: код 204, при повторном GET значение изменилось

5. Удаление параметров (DELETE /user/params)
   i. Ожидаемый результат: код 204, при последующем GET – 404

6.2.3.3. Управление историей веса (User Weight)

1. Создание записи веса (POST /user/weight)
   i. Входные данные: { "weight": 75.5 }
   ii. Ожидаемый результат: код 201, возвращается объект с id и date
2. Создание записи с весом вне диапазона (например, 500)
   i. Ожидаемый результат: код 400
3. Получение последнего веса (GET /user/weight)
   i. Ожидаемый результат: код 200, последняя запись
4. Получение истории с пагинацией (GET /user/weight/history?page=1&limit=10)
   i. Ожидаемый результат: код 200, структура с totalCount и items
5. Удаление записи (DELETE /user/weight/{uuid})
   i. Ожидаемый результат: код 204, запись исчезает из истории

6.2.3.4. Управление калориями (User Calories)

1. Создание записи о калориях (POST /user/calories)
   i. Входные данные: { "calories": 500, "date": "2026-03-20T12:00:00Z", "description": "Обед" }
   ii. Ожидаемый результат: код 201, возвращается id
2. Получение истории за период (GET /user/calories/history?start_date=2026-03-01&end_date=2026-03-20)
   i. Ожидаемый результат: код 200, пагинированный список
3. Обновление записи (PATCH /user/calories/{uuid})
   i. Входные данные: { "calories": 550 }
   ii. Ожидаемый результат: код 204
4. Удаление записи (DELETE /user/calories/{uuid})
   i. Ожидаемый результат: код 204

6.2.3.5. Управление пищевыми записями (Food Entries)

1. Создание записи о приёме пищи (POST /nutrition/entries)

i. Входные данные: { "description": "Яблоко", "calories": 95, "protein": 0.5, "carbs": 25, "fat": 0.3, "meal_type": "snack" }
ii. Ожидаемый результат: код 201, возвращается id

2. Получение дневника за день (GET /nutrition/diary?date=2026-03-20)
   i. Ожидаемый результат: код 200, структура с суммарными показателями и группировкой по приёмам пищи
3. Обновление записи (PATCH /nutrition/entries/{uuid})
   i. Входные данные: { "calories": 100 }
   ii. Ожидаемый результат: код 204
4. Удаление записи (DELETE /nutrition/entries/{uuid})
   i. Ожидаемый результат: код 204
5. Получение отчёта за период (GET /nutrition/report?start_date=2026-03-01&end_date=2026-03-20)
   i. Ожидаемый результат: код 200, агрегированные данные

6.2.3.6. Управление упражнениями (Exercises)

1. Создание упражнения (POST /exercises)
   i. Входные данные: полный JSON с обязательными полями
   ii. Ожидаемый результат: код 201, возвращается id
2. Получение списка с фильтрацией (GET /exercises?level_preparation=beginner&place_exercise=home)
   i. Ожидаемый результат: код 200, только упражнения для начинающих и для дома
3. Получение упражнения по ID (GET /exercises/{uuid})
   i. Ожидаемый результат: код 200, полный объект
4. Обновление упражнения (PATCH /exercises/{uuid})
   i. Входные данные: { "name": "Новое название" }
   ii. Ожидаемый результат: код 204
5. Удаление упражнения (DELETE /exercises/{uuid})
   i. Ожидаемый результат: код 204, при последующем GET – 404

6.2.3.7. Управление задачами (Tasks)

1. Получение списка задач (GET /tasks) с правами администратора.
   i. Ожидаемый результат: код 200, пагинированный список задач
2. Получение задачи по ID (GET /tasks/{uuid})

i. Ожидаемый результат: код 200, детальная информация

3. Перезапуск задачи (POST /tasks/{uuid}/retry)
   i. Ожидаемый результат: код 200 или 202
4. Удаление задачи (DELETE /tasks/{uuid})
   i. Ожидаемый результат: код 204

6.2.3.8. Управление связью тренировок и упражнений (Workout Exercises)

1. Получение списка упражнений для тренировки (GET /workouts/{workoutId}/exercises)
   i. Ожидаемый результат: код 200, массив упражнений с их статусами
2. Обновление статуса упражнения (PATCH /workouts/exercises/{uuid})
   i. Входные данные: { "status": "completed" }
   ii. Ожидаемый результат: код 204

6.2.4. Модуль тренировок (Workout Service)

6.2.4.1. Генерация тренировки по запросу (POST /workouts)

1. Успешная генерация без параметров (на основе профиля)
   i. Входные данные: пустое тело
   ii. Ожидаемый результат: код 201, возвращается WorkoutResponse со списком упражнений (4-12 шт.), полями prediction_calories, duration. Уровень и место соответствуют профилю пользователя
2. Генерация с указанием параметров (например, place_exercise: "gym", exercises_count: 8)
   i. Ожидаемый результат: тренировка содержит упражнения для зала, 8 штук
3. Генерация с неверными параметрами (exercises_count: 25 – больше допустимого)
   i. Ожидаемый результат: код 400
4. Попытка генерации без авторизации
   i. Ожидаемый результат: код 401

6.2.4.2. Получение истории тренировок (GET /workouts/history)

1. Успешное получение списка
   i. Ожидаемый результат: код 200, пагинированный список тренировок
2. Фильтрация по статусу (например, ?status=done)

i. Ожидаемый результат: только выполненные тренировки

6.2.4.3. Получение деталей тренировки (GET /workouts/{uuid})

1. Успешное получение
   i. Входные данные: существующий UUID тренировки
   ii. Ожидаемый результат: код 200, полная информация о тренировке, включая массив упражнений с их статусами
2. Несуществующий UUID
   i. Ожидаемый результат: код 404

6.2.4.4.Обновление статуса тренировки (PATCH /workouts/{uuid})

1. Отметка о выполнении (status: "done", total_calories: 350)
   i. Ожидаемый результат: код 204. При повторном GET статус изменён
2. Попытка обновления не своей тренировки (если реализовано разграничение)
   i. Ожидаемый результат: код 403

6.2.4.5. Удаление тренировки (DELETE /workouts/{uuid})

1. Успешное удаление
   i. Ожидаемый результат: код 204, тренировка исчезает из выборки

6.2.5. Модуль питания (Nutrition Service)

6.2.5.1. Анализ фотографии еды (POST /nutrition/analyze)

1. Успешный анализ (синхронный режим)
   i. Входные данные: файл изображения (JPEG) в multipart/form-data
   ii. Ожидаемый результат: код 200, JSON с полями description, calories, protein, carbs, fat. Запись автоматически сохраняется в дневнике
2. Анализ с некорректным файлом (не изображение)
   i. Ожидаемый результат: код 400
3. Асинхронный режим
   i. Ожидаемый результат: код 202 с task_id. Затем через GET /nutrition/analysis/{taskId} после завершения получаем результат (200) или статус "в обработке" (202)

6.2.6. Модуль рекомендаций (Recommendation Service)

6.2.6.1. Получение списка рекомендаций (GET /recommendations)

1. Успешное получение

i. Ожидаемый результат: код 200, пагинированный список рекомендаций (может быть пустым)

2. Фильтрация по статусу прочтения (?is_read=false)
   i. Ожидаемый результат: только непрочитанные

6.2.6.2. Отметка о прочтении (PATCH /recommendations/{uuid}/read)

1. Успешная отметка
   i. Ожидаемый результат: код 204. При повторном GET рекомендация имеет is_read: true или не показывается в списке активных

6.2.6.3. Ручная генерация рекомендаций (POST /recommendations/refresh)

1. Успешный запуск
   i. Ожидаемый результат: код 202 (если асинхронно) или 200 с сообщением. Через некоторое время новые рекомендации появляются в списке

6.3. Испытание требований к надёжности

6.3.1. Отказоустойчивость при недоступности БД

Временно остановить контейнер PostgreSQL. При попытке выполнения запроса, требующего БД, должен возвращаться код 500 с сообщением о внутренней ошибке. При восстановлении БД работа возобновляется

6.3.2. Обработка повторных попыток в Executor Service

Внести задачу в таблицу tasks с типом email. Настроить так, чтобы отправка не удавалась (например, указать неверные данные провайдера). Убедиться, что задача переходит в статус pending с увеличивающимся attempts и правильным retry_at (экспоненциальная задержка). После исчерпания попыток задача становится failed

6.3.3. Защита от SQL-инъекций

Попытаться передать в параметры запроса значения, содержащие SQL-код (например, в поле username). Система должна обработать их как обычные строки, не вызывая ошибок БД

6.3.4. JWT-безопасность

Попытаться изменить payload токена без знания секрета – токен должен быть отвергнут с кодом 401

ПРИЛОЖЕНИЕ 1

ТЕРМИНОЛОГИЯ

1. **Монолитная архитектура** - архитектурный стиль, при котором все компоненты системы развёртываются как единое целое, выбранный для проекта данного сервиса.
2. **Clean Architecture (Чистая архитектура)** - подход с разделением на слои (domain, application, infrastructure, delivery), использованный для организации кода
3. **DDD (Domain-Driven Design)** - методология проектирования, основанная на выделении доменных моделей (User, Workout, Nutrition, Recommendation).
4. **JWT (JSON Web Tokens)** - стандарт аутентификации с использованием access и refresh токенов.
5. **BCrypt** - алгоритм хеширования паролей, используемый для безопасного хранения учётных данных.
6. **OpenAI API** – апи нейросети
7. **Executor Service** - сервис отложенных задач для обработки уведомлений (push, SMS, email).
8. **S3-совместимое хранилище (MinIO)** - объектное хранилище для изображений (аватары, фото еды).
9. **TDEE (Total Daily Energy Expenditure)** - суточный расход энергии, рассчитываемый по формуле Миффлина-Сан Жеора.
10. **Lifestyle (Образ жизни)** - динамический параметр пользователя (not_active/active/sportive), меняющийся на основе активности.
11. **LevelPreparation (Уровень подготовки)** - показатель физической готовности (beginner/medium/sportsman).
12. **Workout Status (Статус тренировки)** - состояние тренировки: created, in_active, done, failed.
13. **API — это набор правил и протоколов**, который позволяет различным программным компонентам взаимодействовать друг с другом.
14. **Exercise Type (Тип упражнения)** - категоризация упражнений по мышечным группам.
15. **Place Exercise (Место тренировки)** - категория места занятий: home, gym, street.
16. **КБЖУ** - калории, белки, жиры, углеводы; определяемые через анализ питания.
17. **REST API** - архитектурный стиль API, используемый для взаимодействия с клиентом.
18. **PostgreSQL** - реляционная СУБД для хранения всех данных системы.
19. **Redis** - хранилище "ключ-значение" для кэширования данных.
20. **CRUD** - базовые операции Create, Read, Update, Delete для управления данными.

21. **Фоновый сервис** - компонент, выполняющий задачи без прямого запроса пользователя (генерация тренировок).
22. **Push-уведомления** - сообщения, отправляемые на мобильное устройство пользователя.
23. **Промпт (Prompt)** - запрос к OpenAI API для анализа изображений.
24. **Валидация** - проверка корректности входных данных.
25. **Docker** - технология контейнеризации для развёртывания приложения.
26. **CI/CD** - практики непрерывной интеграции и доставки.
27. **Prometheus** - система мониторинга метрик производительности.
28. **Горутина** – легковесный поток выполнения в языке Go, который позволяет конкурентно выполнять функции.
29. **Эндпоинт** – конечная точка API, определяющая конкретный URL и HTTP-метод, по которым клиент может взаимодействовать с сервером.
30. **Промпт -** текстовый запрос, отправляемый к нейросети для получения структурированного ответа.
31. **Метрики** - показатели работы системы (время ответа, частота ошибок).
32. **Генерация тренировок** - автоматическое создание персонализированных программ упражнений.

ПРИЛОЖЕНИЕ 2

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1) ГОСТ 19.101-77 Виды программ и программных документов. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2) ГОСТ 19.102-77 Стадии разработки. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3) ГОСТ 19.103-77 Обозначения программ и программных документов. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4) ГОСТ 19.104-78 Основные надписи. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5) ГОСТ 19.105-78 Общие требования к программным документам. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6) ГОСТ 19.106-78 Требования к программным документам, выполненным печатным способом. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7) ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8) ГОСТ 19.603-78 Общие правила внесения изменений. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9) ГОСТ 19.604-78 Правила внесения изменений в программные документы, выполненные печатным способом. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10) ГОСТ 15150-69 Машины, приборы и другие технические изделия. Исполнения для различных климатических районов. Категории, условия эксплуатации, хранения и транспортирования в части воздействия климатических факторов внешней среды. – М.: Изд-во стандартов, 1997.
11) Устинов В. Надежность оптических дисков: как их правильно хранить и использовать. //Журнал «625» №7. М.: Издательство «625», 2005.
12) ГОСТ Р 7.02-2006 Консервация документов на компакт-дисках. Общие требования. – М.: ИПК Издательство стандартов, 2006.
13) ГОСТ 18300-87 Спирт этиловый ректификованный технический. Технические условия. – М.: ИПК Издательство стандартов, 1997.

14) ГОСТ 9805-84. Спирт изопропиловый. Технические условия. – М.: ИПК Издательство стандартов, 1984.
15) ГОСТ 19.602-78 Правила дублирования, учета и хранения программных документов, выполненных печатным способом. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
16) ГОСТ 19.301-79 Программа и методика испытаний. Требования к содержанию и оформлению. //Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					В сего листов (страниц в докум.)	№ документа	Вхо дящий № сопроводит ельного докум. и дата	одп.	ата
Изм	Изм. ененных	Зам ененных	Нов ых	Ан нулирован ных					